Отчёт по практике 1

Разработка консольного файлового менеджера

Выполнил: Колесников Андрей Юрьевич, 4 курс, группа БСБО-03-23, студенческий билет 23Б0099, номер по журналу 10

РТУ МИРЭА, Институт кибербезопасности и цифровых технологий

1. Цель работы

Нужно было сделать консольный файловый менеджер на C#/.NET 8. Программа должна показывать диски и содержимое каталогов, создавать, читать, изменять и удалять файлы, работать с JSON/XML и ZIP, хранить пользователей и информацию о файлах в базе данных, а все действия записывать в журнал. Отдельное требование — регистрация и вход, разграничение доступа к файлам по владельцу и защита от SQL-инъекций.

2. Стенд и структура проекта

Проект собран под .NET 8, внешних NuGet-пакетов нет: в каталоге лежит NuGet.Config с очищенным списком источников, а SQLite вызывается через P/Invoke. Это позволило не зависеть от интернета при сборке и показать работу с нативной библиотекой. На Windows используется системная winsqlite3.dll, на Linux — libsqlite3.so.0, на macOS — системная libsqlite3.dylib. Библиотека выбирается в SqliteNative.cs через NativeLibrary.SetDllImportResolver, поэтому один и тот же код работает на Windows и в Linux-контейнере.

Файл	Что делает
Program.cs	меню, регистрация/вход, режим --selfcheck
AuthService.cs	PBKDF2-SHA256, соль, проверка пароля
Db.cs	подключение к SQLite, схема и параметризованные запросы
SqliteNative.cs	P/Invoke и выбор библиотеки под ОС
FileOpsService.cs	создание, чтение, изменение, удаление, аудит
JsonXmlService.cs	экспорт и импорт JSON/XML
ZipService.cs	создание и безопасная распаковка ZIP
DiskService.cs	список дисков
Models.cs	пользователь и запись файла

2.1. Соответствие требованиям

Требование задания	Где реализовано	Чем подтверждается
Список дисков и свойства файловой системы	DiskService.cs, DriveInfo	меню 1: тип, файловая система, метка, всего/свободно МБ

Чтение, запись и удаление файлов	FileOpsService.cs	меню 2–5; проверки в --selfcheck
JSON и XML: хранение и чтение	JsonXmlService.cs	меню 6–9; импорт и экспорт в selfcheck
ZIP: создание и извлечение	ZipService.cs	меню 10–11; проверка Zip Slip
База данных: файлы и операции	Db.cs, SQLite	таблицы Users, Files, Operations
Журнал действий пользователей	Operations + LogOperation	меню 12; записи после каждого действия
Аутентификация и контроль доступа	AuthService.cs, owner_id	PBKDF2, проверка владельца файла
Подготовленные выражения	Db.Prepare, sqlite3_bind_*	все запросы с параметрами; SQLi не проходит

2.2. Список дисков

Пункт меню 1 обходит DriveInfo.GetDrives() и для каждого готового диска печатает тип, файловую систему, метку тома, общий объём и доступное место. Если диск не готов, остальные свойства не читаются, чтобы не получить исключение на пустом CD-приводе.

Свойство	Источник	Что показывает
Имя / точка монтирования	DriveInfo.Name	C:\, / и т.п.
Тип	DriveInfo.DriveType	Fixed, Removable, Network
Файловая система	DriveInfo.DriveFormat	NTFS, ext4, APFS
Метка тома	DriveInfo.VolumeLabel	имя тома
Всего места	DriveInfo.TotalSize	в МБ
Свободно	DriveInfo.AvailableFreeSpace	в МБ

3. База данных и разграничение доступа

База SQLite создаётся автоматически при первом запуске. В ней три таблицы: Users, Files и Operations. У Files есть owner_id, поэтому при чтении, изменении и удалении проверяется, что файл принадлежит текущему пользователю. В Operations пишутся время, тип операции, ссылки на файл и пользователя и подробности — создание, чтение, изменение, удаление, экспорт, импорт, архивирование и распаковка.

Таблица	Поля
Users	id, username (UNIQUE), password_hash, salt, created_at
Files	id, filename, created_at, size, location, owner_id
Operations	id, op_time, operation_type, file_id, user_id, details

Пароль не хранится в открытом виде: AuthService генерирует 16-байтную соль и считает PBKDF2-SHA256 с 100 000 итераций, результат — 32 байта. Сравнение хэшей

идёт через FixedTimeEquals, чтобы не давать подсказку по времени. Для несуществующего пользователя всё равно выполняется проверка с заглушкой, поэтому по времени ответа нельзя определить, есть ли такой логин.

Все SQL-запросы идут через sqlite3_prepare_v2 и привязку параметров. Имя файла, логин и другие пользовательские строки никогда не склеиваются с текстом запроса. Это закрывает SQL-инъекцию (CWE-89). Включены PRAGMA foreign_keys=ON и busy_timeout=5000, чтобы ссылки оставались целостными, а второй экземпляр программы не падал сразу на блокировке.

4. Файлы, JSON/XML и ZIP

FileOpsService ограничивает чтение файла 10 МБ, использует FileShare.None на время работы и пишет каждую операцию в журнал. JsonXmlService принимает файлы до 5 МБ и до 10 000 записей, для JSON ограничивает глубину 16, для XML запрещает DTD и внешние сущности (DtdProcessing.Prohibit, XmlResolver = null). Импорт не выполняет код из файла, а разбирает данные в объекты и затем пишет их в БД параметрами.

ZipService при распаковке проверяет каждый путь: запрещены .., абсолютные пути и выход за каталог назначения. Дополнительно ограничены число записей (5 000), размер одной записи (200 МБ) и общий размер (500 МБ), а также коэффициент сжатия 1 000:1 — это защита от zip-бомбы. Байты считаются фактически при распаковке, а не по заголовку архива; при ошибке частично распакованные файлы удаляются.

5. Проверка

Сборка и самопроверка запускаются командами:

```
dotnet build -v q
dotnet bin/Debug/net8.0/FileManagerApp.dll --selfcheck
```

Сборка завершается без предупреждений и ошибок. --selfcheck выполняет 18 проверок на временной папке и в конце печатает «все проверки пройдены успешно». Проверяются регистрация и вход, неверный пароль, короткий пароль, создание/чтение/изменение/удаление, запрет чтения чужого файла, экспорт и импорт JSON и XML, создание и распаковка ZIP, блокировка Zip Slip и записи в журнале.

Что проверено	Результат
dotnet build	0 предупреждений, 0 ошибок
--selfcheck	18 из 18, итог «все проверки пройдены успешно»
SQLite на Windows	winsqlite3.dll, без установки сторонних пакетов
SQLite на Linux	libsqlite3.so.0, тот же код
Ошибка входа	общее сообщение без раскрытия логина

6. Ограничения

База не шифруется, это локальный учебный вариант. Символические ссылки отдельно не разыменовываются: при необходимости их стоит проверять через File.ResolveLinkTarget. Хранимых процедур в SQLite нет, поэтому вместо них используются подготовленные выражения — ближайший аналог по смыслу. Приложение консольное, многопользовательская работа ограничена локальным файлом БД и настройками блокировок SQLite.

7. Выводы

Получился рабочий файловый менеджер с разделением на модули, собственной аутентификацией, аудитом и безопасной работой с архивами. Главное, что проверялось: SQL-инъекция не проходит из-за параметров, пароли не лежат открытым текстом, чужой файл не читается, ZIP не выходит за каталог распаковки. 18 самопроверок проходят, сборка чистая; этот же проект используется в практиках 2 и 5 как тестовое приложение.